Building a Mobile App with Xamarin

Xamarin is a tool used for cross-platform mobile app development that allows developers to share a majority of the code across major platforms such as Android, iOS, Windows and macOS. This article is a tutorial on how you can use Xamarin to deliver a native Android app.

Xamarin has been built by the engineers who created Mono, the .NET Framework based open source development platform (Miguel de Icaza led the efforts to develop Mono). The Xamarin company was founded in May 2011, and Microsoft acquired it in 2016.

Xamarin uses a single language, i.e., C# to create apps for mobile platforms. It is natively compiled unlike other cross-platform solutions, which makes it a go-to option for building high-performance apps with a native look and feel. Additionally, Xamarin can hold all the native and the latest API access to utilise the underlying platform's capabilities, such as multi-window on Android and ARKit on iOS.

In Xamarin, platforms can share the code related to business logic, database layer and network communication to create platform-specific UI layers for a better user experience. Thus, Xamarin apps look the same as native apps on any device, as compared to generic hybrid apps.

Why use Xamarin

Earlier, mobile application development used to be limited, with fewer devices running on platforms with specific languages. But today, with the increased range of mobile devices and mobile company based OSs, building an application for each platform implies additional development and maintenance costs. Xamarin development is not a 'one size fits all' concept; it's about increasing productivity while incurring minimal cost.

The top five benefits of Xamarin are:

- A single technology stack for all
- Performance that is the same as native app development
- Native user experience
- Open source
- Easy to maintain

There are mainly two methods of building mobile applications using Xamarin:

- Xamarin Native
- Xamarin Forms

Xamarin Native uses Xamarin.Android and Xamarin.iOS to compile the source code. For iOS, it uses ahead-of-time compilation, while for Android, just-in-time compilation is done. In both the cases, the process takes care of memory allocation, garbage collection and platform interoperability issues, by default.